COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	8/11/26	**DATE SUBMITTED:**	8/11/26
NAME:	Mike Justin A. Cabangon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 QUEUES

1. PROCEDURE OUTPUT

ILO A: main.cpp

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main(){

    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');

    //display
    display(q);
    //empty()
    if(q.empty()){
        std::cout<<"Queue ie empty"<<"\n";
    }
        else {
            std::cout<<"Queue is not empty"<<"\n";
        }

    //size()
    std::cout<<"Queue size: "<<q.size()<<"\n";
    //front()
    std::cout<<"Front of the queue: "<<q.front()<<"\n";
```

```
31      //back
32      std::cout<<"Back of the queue: "<<q.back()<<"\n";
33      //pop
34      q.pop(); std::cout<<"Popping out the front element"<<"\n";
35      //display the new front
36      std::cout<<"Front of the queue: "<<q.front()<<"\n";
37      //display
38      display(q);
39      //push another element
40      q.push('1'); std::cout<<"Pushing 1 at the rear"<<"\n";
41      //display the new back
42      std::cout<<"Back of the queue: "<<q.back()<<"\n";
43      //display
44      display(q);
45
46  }
47
48  void display(std::queue<char> r){
49      //create a copy of the queue
50      std::queue<char> c = r;
51
52      while(!c.empty()){
53          std::cout<<" "<<c.front();
54          c.pop();
55
56      }
57      std::cout<<"\n";
58  }
```

OUTPUT

```
Queue is not empty
Queue size: 6
Front of the queue: c
Back of the queue: 0
Popping out the front element
Front of the queue: p
 p e 0 1 0
Pushing 1 at the rear
Back of the queue: 1
 p e 0 1 0 1
```

ILO B1: main.cpp

```cpp
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue(&front, &back, 'M');
    std::cout<<front->data<< " "<<back->data<<"\n";
    enqueue(&front, &back, 'I');
    std::cout<<front->data<< " "<<back->data<<"\n";
    enqueue(&front, &back, 'K');
    std::cout<<front->data<< " "<<back->data<<"\n";
    enqueue(&front, &back, 'E');
    std::cout<<front->data<< " "<<back->data<<"\n";
    dequeue(&front, &back);
    std::cout<<front->data<< " "<<back->data<<"\n";

    std:: cout << "Queue list \n";
    display(front);
    std::cout << "Front value: " << FrontVariable(front) << std::endl;

    return 0;


}

```

ILO B1: queuell.h

```
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
    public:
    T data;
    qNode *next;
};
//creating a new node
template <typename T>
qNode<T> *new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}
template<typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    //create a new node
    qNode<T>* newNode = new_node(newData);

    //inserting to an empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
    (*frontPtr) = newNode;
    (*backPtr) = newNode;
}
    //inserting an item into a non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
```

```
31
32  }
33
34  template <typename T>
35  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
36      //create a temporary node to store the node to be deleted
37      qNode<T>* deleteNode = nullptr;
38      deleteNode = (*frontPtr);
39      //check if the queue is only 1 node
40      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
41          (*frontPtr) = nullptr;
42          (*backPtr) = nullptr;
43          delete deleteNode;
44          return;
45      }
46
47      //deleting of the node
48      (*frontPtr)=deleteNode->next;
49      delete deleteNode;
50
51
52  }
53  //display all elements in the list
54  template <typename T>
55  void display(qNode<T>* frontPtr){
56     if(frontPtr == nullptr){
57        std::cout<< "queue is empty \n";
58        return;
59     }
60
```

```
61      qNode<T>* current = frontPtr;
62      while (current != nullptr){
63       std::cout << current->data << "->";
64       current= current->next;
65      }
66      std::cout<< "NULL \n";
67
68  }
69
70  //return the front variable
71  template <typename T>
72  T FrontVariable(qNode<T>* frontPtr){
73      if(frontPtr != nullptr) {
74       return frontPtr->data;
75      }
76  }
77
78
79  #endif
```

OUTPUT

```
M M
M I
M K
M E
I E
Queue list
I->K->E->NULL
Front value: I
```

ILO B2: main.cpp

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    //create an objecy
    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(2);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(3);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(4);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(5);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(6);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;


    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

```

```cpp
    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    //copy constructor
    queueArr<int> q2(q);

    std::cout << "Copy constructor test\n" << std::endl;
    std::cout << "q2_front: " << q2.Front()
              << ", q2_back: " << q2.Back()
              << ", q2_size: " << q2.size() << std::endl;

    // copy assignment operator
    queueArr<int> q3(3);

    q3 = q;

    std::cout << "Copy assignment" << std::endl;
    std::cout << "q3_front: " << q3.Front()
              << ", q3_back: " << q3.Back()
              << ", q3_size: " << q3.size() << std::endl;

    return 0;
}
```

ILO B2: QueueArr.h

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

template <typename T>
class queueArr{
    private:
        //create a pointer array
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        //prototypes to use for queueArr:
        queueArr(size_t capacity);
        bool empty();
        bool full();

        size_t size();
        void Clear();

        T Front();
        T Back();

        void Enqueue(T newData);
        void Dequeue();

        //tasks
        // destructor
        ~queueArr();

        // copy constructor
```

```
36          queueArr(const queueArr& other);
37
38          //copy assignment operator
39          queueArr& operator=(const queueArr& other);
40  };
41
42
43  //constructor
44  template <typename T>
45  queueArr<T>::queueArr(size_t capacity){
46
47      //capacity of the array
48      q_capacity = capacity;
49
50      //current size of the array
51      q_size = 0;
52
53      //initial front and back of the index
54      q_front = 0;
55      q_back = -1;
56
57      //newly allocated array
58      q_array = new T[q_capacity];
59      std::cout << "Queue created with capacity: "
60                << q_capacity << std::endl;
61  }
62
63  //copy constructor
64  template <typename T>
65  queueArr<T>::queueArr(const queueArr& other){
66      q_capacity = other.q_capacity;
67      q_size = other.q_size;
68      q_front = other.q_front;
69      q_back = other.q_back;
70
```

```
71      q_array = new T[q_capacity];
72
73      for (size_t i = 0; i < q_capacity; i++){
74          q_array[i] = other.q_array[i];
75      }
76
77      std::cout << "Copy constructor called." << std::endl;
78  }
79
80
81  //copy assignment operator
82  template <typename T>
83  queueArr<T>& queueArr<T>::operator=(const queueArr& other){
84      if (this != &other){
85
86          delete [] q_array;
87
88          q_capacity = other.q_capacity;
89          q_size = other.q_size;
90          q_front = other.q_front;
91          q_back = other.q_back;
92
93          q_array = new T[q_capacity];
94
95          for (size_t i = 0; i < q_capacity; i++){
96              q_array[i] = other.q_array[i];
97          }
98      }
99
100     std::cout << "Copy assignment operator called." << std::endl;
101
102     return *this;
103 }
104
```

```cpp
//delete
template <typename T>
queueArr<T>::~queueArr(){
    delete [] q_array;
    std::cout << "Queue destroyed." << std::endl;
}

//empty
template <typename T>
bool queueArr<T>::empty(){
    return q_size == 0;
}

//full
template <typename T>
bool queueArr<T>::full(){
    return q_size == q_capacity;
}

//clear
template <typename T>
void queueArr<T>::Clear(){
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

//return the element in the front of the queue
template <typename T>
T queueArr<T>::Front(){
    if (empty()){
        std::cout << "Queue is empty." << std::endl;
        return T{}; // return default value of T
    }
```

```cpp
    //return the front element
    return q_array[q_front];
}

// back
template <typename T>
T queueArr<T>::Back(){
    //check and display if the queue is empty
    if (empty()){
        std::cout << "Queue is empty." << std::endl;
        return T{}; // return the default value of T
    }
    // if not then return the back element
    return q_array[q_back];
}

template <typename T>
size_t queueArr<T>::size(){
    return q_size;
}

template <typename T>
void queueArr<T>::Enqueue(T newData){
    if (full()){
        std::cout << "Queue is full." << std::endl;
        return;
    }

    //compute for the q_back to move it circularly
    q_back = (q_back + 1) % q_capacity;

    //assign the new value to the back of the queue array
    q_array[q_back] = newData;

```

```
173     //increment the q_size
174     q_size++;
175 }
176
177 //dequeue
178 template <typename T>
179 void queueArr<T>::Dequeue(){
180     //check if the queue is empty
181     if (empty()){
182         std::cout << "Queue is empty." << std::endl;
183         return;
184     }
185
186     //compute for q_front to move it circularly
187     q_front = (q_front + 1) % q_capacity;
188
189     //decrement the size q_size
190     q_size--;
191 }
192
193 #endif
```

OUTPUT

```
Queue created with capacity: 6
q_front: 1, q_back: 1, q_size: 1
q_front: 1, q_back: 2, q_size: 2
q_front: 1, q_back: 3, q_size: 3
q_front: 1, q_back: 4, q_size: 4
q_front: 1, q_back: 5, q_size: 5
q_front: 1, q_back: 6, q_size: 6
2, q_back: 6, q_size: 5
3, q_back: 6, q_size: 4
4, q_back: 6, q_size: 3
5, q_back: 6, q_size: 2
6, q_back: 6, q_size: 1
Queue is empty.
0, q_back: Queue is empty.
0, q_size: 0
Queue is empty.
Queue is empty.
0, q_back: Queue is empty.
0, q_size: 0
Copy constructor called.
Copy constructor test

q2_front: Queue is empty.
0, q2_back: Queue is empty.
0, q2_size: 0
Queue created with capacity: 3
Copy assignment operator called.
Copy assignment
q3_front: Queue is empty.
0, q3_back: Queue is empty.
0, q3_size: 0
Queue destroyed.
Queue destroyed.
Queue destroyed.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: supp.cpp

5p1 supp > supp.cpp > main()

```cpp
#include <iostream>
#include "Jobsupp.h"
#include "Printersupp.h"

int main() {

    // create objext
    Printer printer;

    // add jobs to printer line
    printer.addJob(Job(1, "Mike", 5));
    printer.addJob(Job(2, "Corro", 10));
    printer.addJob(Job(3, "Jave", 3));
    printer.addJob(Job(4, "Dane", 7));

    // display jobs
    printer.displayJobs();

    // process jobs
    std::cout << "Processing Jobs\n" << std::endl;

    printer.processJob();
    printer.processJob();
    printer.processJob();
    printer.processJob();

    // try to process another job when the queue is empty
    printer.processJob();

    return 0;
}
```

ILO C:Jobsupp.h

5p1 supp > C Jobsupp.h > ...

```
#ifndef JOBsupp_H
#define JOBsupp_H

#include <iostream>
#include <string>

class Job {
private:
    int id;
    std::string user;
    int pages;

public:

    // Default constructor
    Job() {
        id = 0;
        user = "";
        pages = 0;
    }

    // Constructor to create a job
    Job(int id, std::string user, int pages) {
        this->id = id;
        this->user = user;
        this->pages = pages;
    }

    // Display the information of the job
    void display() {
        std::cout << "Job ID: " << id
                  << ", User: " << user
                  << ", Pages: " << pages << std::endl;
    }
};

#endif
```

ILO C: Printersupp.h

```
#ifndef PRINTERsupp_H
#define PRINTERsupp_H

#include <iostream>
#include "Jobsupp.h"

class Printer {
private:
    // store jobs
    Job jobs[10];

    int front;
    int back;
    int size;

public:

    //constructor
    Printer() {
        front = 0;
        back = -1;
        size = 0;
    }

    //add new job
    void addJob(Job newJob) {

        //if full
        if (size == 10) {
            std::cout << "Printer queue is full." << std::endl;
            return;
        }

        //move back to next position
        back = (back + 1) % 10;

```

```
37          // store new job at the back
38          jobs[back] = newJob;
39
40          // increase jobs
41          size++;
42
43          std::cout << "Job added to printer queue." << std::endl;
44      }
45
46      // process first job in line
47      void processJob() {
48
49          // if no jobs
50          if (size == 0) {
51              std::cout << "No jobs waiting." << std::endl;
52              return;
53          }
54
55          // display jobs
56          std::cout << "Processing job:\n" << std::endl;
57          jobs[front].display();
58
59          // move front to the next job
60          front = (front + 1) % 10;
61
62          // decrease jobs
63          size--;
64      }
65      // display pending jobs
66      void displayJobs() {
67
68          // if queue empty
69          if (size == 0) {
70              std::cout << "No pending jobs." << std::endl;
```

```
71              return;
72          }
73
74          std::cout << "Pending jobs:\n" << std::endl;
75
76          // go through jobs in queue
77          for (int i = 0; i < size; i++) {
78
79              // position of job
80              int index = (front + i) % 10;
81
82              //display job
83              jobs[index].display();
84          }
85      }
86  };
87  #endif
```

OUTPUT

```
Job added to printer queue.
Job added to printer queue.
Job added to printer queue.
Job added to printer queue.
Pending jobs:

Job ID: 1, User: Mike, Pages: 5
Job ID: 2, User: Corro, Pages: 10
Job ID: 3, User: Jave, Pages: 3
Job ID: 4, User: Dane, Pages: 7
Processing Jobs

Processing job:

Job ID: 1, User: Mike, Pages: 5
Processing job:

Job ID: 2, User: Corro, Pages: 10
Processing job:

Job ID: 3, User: Jave, Pages: 3
Processing job:

Job ID: 4, User: Dane, Pages: 7
No jobs waiting.
```

WHY I USED ARRAY?: I used an array because it is simple and easy to manage for storing the printer jobs. It also works well with using the FIFO principle in queue, where the first job added is the first job processed.

Output Analysis:
The output shows that the printer adds jobs to the queue and processes them in the same order they were added just like falling in line. It is because the queue uses FIFO (First In, First Out) principle. Once all jobs are processed, the printer shows that there are no more jobs waiting.

5. Conclusions/Learnings

Provide the following:

- **Summary of lessons learned**

I learned how queues work using the FIFO principle and mostly about how to implement queue operations using arrays.

- **Analysis of the procedure**

The procedure helped me understand how to add, remove and access elements in a queue using different operations such as enqueue and dequeue.

- **Analysis of the supplementary activity**

With the lessons I learned, I used the array to apply in a situation of a shared printer, and I used the FIFO principle. The jobs were processed in the same order they were submitted.

- **Concluding statement / Feedback: How well did you think you did in this activity? What are your areas forimprovement?**

I feel little to no confidence because I had a lot of trouble understanding the lines of code and took too much time completing the activity. Concludes that I still need to improve my understanding and master the concepts of programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.